COMPUTER ENGINEERING
QUEZON CITY

Final Project: Barangay Document Generator

Course Code: CPE 010 **Course Title: Data Structures and Algorithms**

PROJECT INFORMATION

How to use this template: text in gray italics is a guide; delete it before submitting. Replace everything in [square brackets] with your own content. Keep the section numbers and headings. Number figures and tables in order (Figure 1, Table 1, ...) and refer to them in your text.

Item	Details
Project Title	[Short, descriptive name of the system]
Business Process Emulated	[e.g., Hospital patient admission and queueing]
Section / Group No.	CPE21S2 / Team 1
Instructor	Engr. Jimlord M. Quejado
Date Submitted	November 6, 2026
GitHub Repository	[https://github.com/...] (details in Appendix C)

Group members, roles, and contributions are listed in **Appendix B. Team Composition**.

Intended Learning Outcomes Addressed

At the end of this activity, the students should be able to:

- Apply fundamental data structures and techniques to solve engineering-related problems through algorithm design.
- Design efficient algorithms and analyze their complexity.
- Build efficient algorithms based on selected data structures using good programming styles in C++.

Objectives

State one general objective, then three to five specific objectives. Each specific objective should name the business task and the data structure or algorithm that supports it, and should be checked in Sections 4 and 5.

General Objective: [To develop a C++ program that emulates the ... process of ... using appropriate data structures and algorithms.]

Specific Objectives:

1. [To manage ... using a queue so that ... are served in first-come, first-served order.]
2. [To search ... records efficiently using binary search.]
3. [To sort ... reports by ... using ... sort.]
4. [To analyze the time complexity of the main operations theoretically and experimentally.]

1. BUSINESS PROCESS

1.1 Organization and Business Process

Name the real or realistic organization (a bank, clinic, school registrar, warehouse, delivery service, restaurant, parking facility) and describe the business process your system emulates: who is involved, what goes in, and what comes out. One to two paragraphs.

[Write here.]

1.2 Current Process (As-Is)

List the steps of the process as it is done today, in order. Identify who performs each step.

Step	Activity	Performed By	Data Involved
1	[e.g., Customer gets a queue number]	[Customer]	[Name, service type]
2	[ ]	[ ]	[ ]
3	[ ]	[ ]	[ ]
4	[ ]	[ ]	[ ]

1.3 Problems in the Current Process

Explain what is slow, error-prone, or hard to manage in the current process, with concrete examples (long waiting time, lost records, slow searches, wrong priority).

1. [Problem 1]
2. [Problem 2]
3. [Problem 3]

1.4 Proposed System

Describe what your program does to emulate and improve the process. List its main features.

#	Feature	Business Task It Supports
1	[e.g., Add customer to the waiting line]	[Queue management]
2	[ ]	[ ]
3	[ ]	[ ]
4	[ ]	[ ]
5	[ ]	[ ]

1.5 Scope and Delimitations

State what the program covers and what it does not (for example: console program only, data kept in memory or text files, single user, no network or database).

- [The system covers ...]
- [The system does not ...]

2. DATA STRUCTURES AND ALGORITHMS APPLIED

Discuss at least FIVE data structure or algorithm topics from the course and how each is applied in your system. This section is DISCUSSION ONLY: do not include code here. Explain the concept in your own words, connect it to a specific part of the business process, and justify why it is the right choice. Refer to Appendix A for the implementation.

Course topics you may choose from: arrays and dynamic memory allocation; singly, doubly, and circular linked lists; stacks (array or linked-list based); queues (array or linked-list based); linear search; binary search; hashing; sorting algorithms; general trees and binary search trees; AVL trees; tree traversal algorithms; graphs (adjacency matrix or adjacency list); breadth-first search; depth-first search.

2.1 Summary of Data Structures and Algorithms Applied

#	Data Structure / Algorithm	Where It Is Used in the Business Process	Main Operations
1	[e.g., Queue (linked list)]	[Customers waiting to be served]	[enqueue, dequeue, peek]
2	[ ]	[ ]	[ ]
3	[ ]	[ ]	[ ]
4	[ ]	[ ]	[ ]
5	[ ]	[ ]	[ ]

2.2 [Data Structure / Algorithm 1]

Concept: [Explain how this data structure or algorithm works, in your own words. A simple diagram is encouraged.]

Application in the System: [Which business task uses it, what data it stores or processes, and what happens step by step.]

Justification: [Why this choice is better than an alternative for this task, e.g., why a queue instead of an array, or binary search instead of linear search.]

Operations and Complexity: [Main operations used and their time complexity, e.g., enqueue O(1), search O(log n). Full analysis goes in Section 5.]

2.3 [Data Structure / Algorithm 2]

Concept: [Explain how this data structure or algorithm works, in your own words. A simple diagram is encouraged.]

Application in the System: [Which business task uses it, what data it stores or processes, and what happens step by step.]

Justification: [Why this choice is better than an alternative for this task, e.g., why a queue instead of an array, or binary search instead of linear search.]

Operations and Complexity: [Main operations used and their time complexity, e.g., enqueue O(1), search O(log n). Full analysis goes in Section 5.]

2.4 [Data Structure / Algorithm 3]

Concept: [Explain how this data structure or algorithm works, in your own words. A simple diagram is encouraged.]

Application in the System: [Which business task uses it, what data it stores or processes, and what happens step by step.]

Justification: [Why this choice is better than an alternative for this task, e.g., why a queue instead of an array, or binary search instead of linear search.]

Operations and Complexity: [Main operations used and their time complexity, e.g., enqueue O(1), search O(log n). Full analysis goes in Section 5.]

2.5 [Data Structure / Algorithm 4]

Concept: [Explain how this data structure or algorithm works, in your own words. A simple diagram is encouraged.]

Application in the System: [Which business task uses it, what data it stores or processes, and what happens step by step.]

Justification: [Why this choice is better than an alternative for this task, e.g., why a queue instead of an array, or binary search instead of linear search.]

Operations and Complexity: [Main operations used and their time complexity, e.g., enqueue O(1), search O(log n). Full analysis goes in Section 5.]

2.6 [Data Structure / Algorithm 5]

Concept: [Explain how this data structure or algorithm works, in your own words. A simple diagram is encouraged.]

Application in the System: [Which business task uses it, what data it stores or processes, and what happens step by step.]

Justification: [Why this choice is better than an alternative for this task, e.g., why a queue instead of an array, or binary search instead of linear search.]

Operations and Complexity: [Main operations used and their time complexity, e.g., enqueue O(1), search O(log n). Full analysis goes in Section 5.]

Add sub-sections 2.7, 2.8, ... if you apply more than five topics.

3. SYSTEM FLOWCHART

Draw a basic flowchart of the whole system: from program start, through the main menu and each major operation, to program end. Use standard flowchart symbols and label every decision branch (Yes / No). You may use draw.io, Lucidchart, or Microsoft Visio. Add separate flowcharts for complex operations if needed.

Symbol	Name	Use

Oval	Terminator	Start and End
Rectangle	Process	An action or computation, e.g., "Add patient to queue"
Parallelogram	Input / Output	Reading input or displaying output
Diamond	Decision	A yes/no question, e.g., "Is the queue empty?"
Rectangle with double sides	Predefined Process	An operation detailed in another flowchart
Arrow	Flow Line	The order of steps

3.1 Main System Flowchart

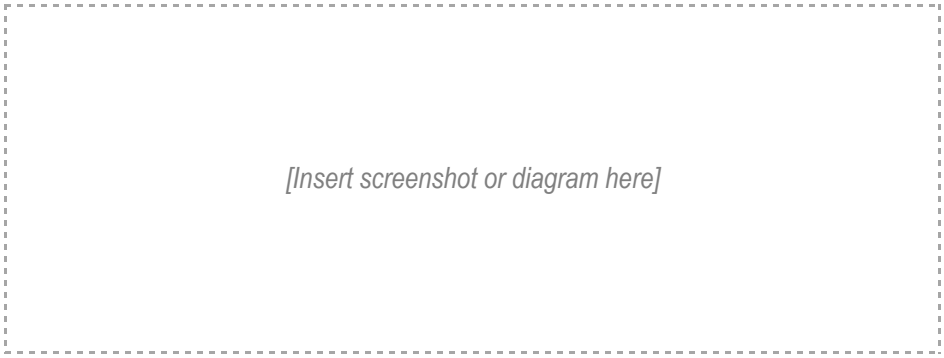


Figure 1. Flowchart of [Project Title]

3.2 Flow Description

Explain the flowchart in numbered steps, matching the labels in Figure 1.

1. [The program starts and loads ...]
2. [The main menu displays ...]
3. [ ]

4. OUTPUT

Show the program running for every feature in Section 1.4, including at least one invalid input and how the program handles it. Use screenshots of the actual console or window output with readable text. Below each figure, explain what input was given and what the output shows. The complete source code is in Appendix A.

Feature	Figure No.	Input Used
[Feature 1]	[Figure 2]	[ ]
[Feature 2]	[Figure 3]	[ ]
[Invalid input handling]	[Figure 4]	[ ]

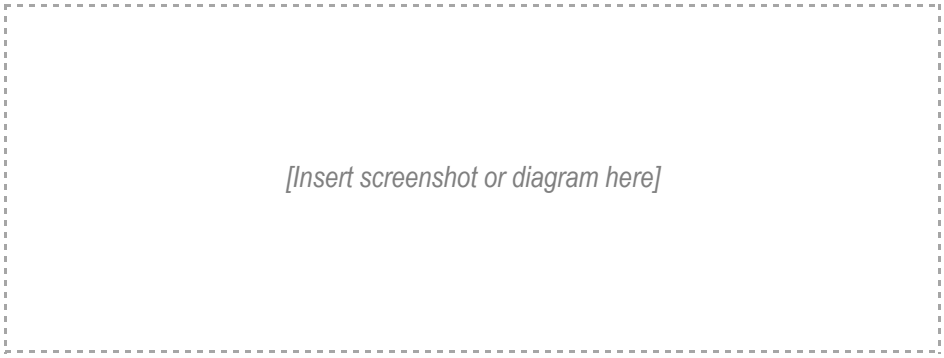


Figure 2. [Main menu] - [what it shows]

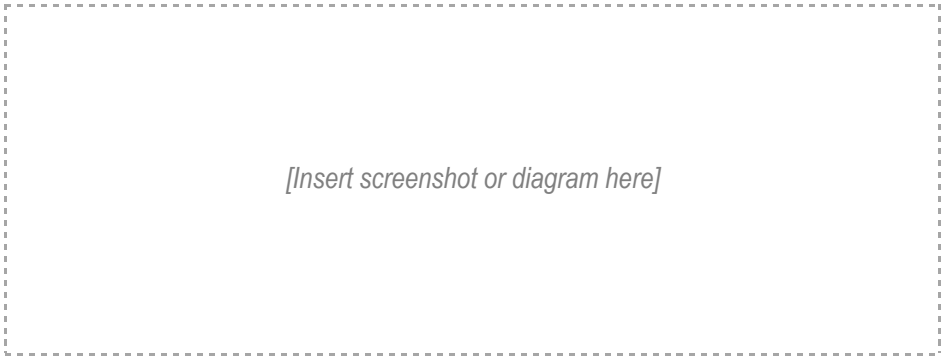


Figure 3. [Feature output] - [what it shows]

Add more figures as needed.

5. PERFORMANCE ANALYSIS

Analyze the efficiency of your program in two ways: theoretically (Big-O, from the algorithm) and experimentally (measured running time on real inputs). Then compare the two.

5.1 Theoretical Analysis

For each main operation, give the best, average, and worst-case time complexity and the space complexity, and briefly explain how each was derived (for example, count how many times the main loop runs in terms of n).

Operation	Data Structure / Algorithm	Best	Average	Worst	Space
[e.g., Add customer]	[Queue]	[O(1)]	[O(1)]	[O(1)]	[O(1)]
[e.g., Search record]	[Hash table]	[O(1)]	[O(1)]	[O(n)]	[O(n)]
[e.g., Sort report]	[ ]	[ ]	[ ]	[ ]	[ ]
[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Derivation: [Explain how the complexity of at least two operations was obtained.]

5.2 Experimental Analysis

Measure the actual running time of at least two operations for increasing input sizes. Run each measurement several times and report the average. Describe your test setup so the results can be repeated.

Setup Item	Details
Computer (CPU, RAM)	[e.g., Intel Core i5-1135G7, 8 GB RAM]
Operating system	[e.g., Windows 11]
Compiler and settings	[e.g., g++ 13.2, -O2]
Timing method	[e.g., std::chrono::steady_clock around the operation only]
Input data	[e.g., randomly generated records; same data for every algorithm]
Repetitions	[e.g., 5 runs per input size; average reported]

Input Size (n)	[Operation 1] Time (ms)	[Operation 2] Time (ms)	[Operation 3] Time (ms)
100	[ ]	[ ]	[ ]
1,000	[ ]	[ ]	[ ]
10,000	[ ]	[ ]	[ ]
100,000	[ ]	[ ]	[ ]

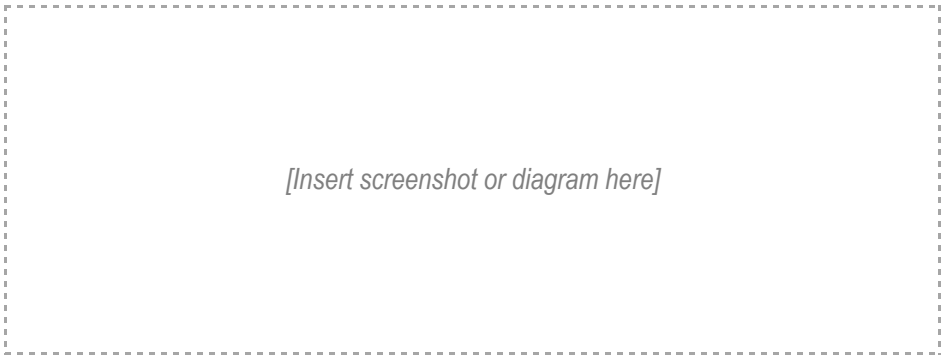


Figure 5. Running time vs. input size for [operations]

Plot the measured times as a line graph with input size n on the x-axis and time on the y-axis, one line per operation.

5.3 Comparison of Theoretical and Experimental Results

Do the measured times grow the way Big-O predicts? For example, when n increases 10 times, an O(n) operation should take about 10 times longer, an O(n log n) operation slightly more than 10 times, and an O(n²) operation about 100 times longer. Explain any differences (small inputs, caching, system load, constant factors).

Operation	Expected Growth (Big-O)	Observed Growth	Agrees? (Yes / No)
[ ]	[ ]	[e.g., about 10x per 10x increase in n]	[ ]
[ ]	[ ]	[ ]	[ ]

[Write your discussion here.]

6. CONCLUSION AND RECOMMENDATIONS

6.1 Conclusion

Summarize how the system emulates the business process, whether the objectives were met, and what the performance analysis showed.

[Write here.]

6.2 Recommendations

Suggest specific improvements, such as a more efficient data structure for a slow operation.

- [Recommendation 1]
- [Recommendation 2]

7. REFERENCES

List every source used in APA 7th edition, in alphabetical order. Under the course AI policy, AI tools may be used only with prior approval from the instructor, and any use must be fully disclosed here: the tool, what it was used for, and which parts of the project it affected.

[Author, A. A. (Year). Title of work. Publisher. URL]

APPENDICES

Appendix A. Source Code

Paste the complete C++ code of every file as text (not screenshots), in a monospace font such as Consolas or Courier New, 8 to 9 pt, keeping the indentation. Include header files (.h) and source files (.cpp), and the code used for the experimental timing.

A.1 main.cpp

```
// [Screenshot the full contents of main.cpp here]
```

A.2 [file_name.h / file_name.cpp]

```
// [Screenshot the full contents of main.cpp here]
```

A.3 [timing / benchmark code]

```
// [Screenshot the full contents of main.cpp here]
```

Appendix B. Team Composition

#	Name (Surname, First Name M.I.)	Student No.	Role	Main Contributions	%
1	[ ]	[ ]	[e.g., Project leader]	[e.g., Queue module, flowchart]	[ ]
2	[ ]	[ ]	[ ]	[ ]	[ ]
3	[ ]	[ ]	[ ]	[ ]	[ ]
4	[ ]	[ ]	[ ]	[ ]	[ ]

Appendix C. GitHub Repository

The repository must be public or shared with the instructor, and must contain all source files and a README. Every member should have commits in the history.

Item	Details
Repository link	[https://github.com/...]
Branch / release submitted	[e.g., main, tag v1.0]
README contents	[Project description, how to compile and run, sample input, team members]
How to compile	[e.g., g++ -std=c++17 -O2 main.cpp queue.cpp -o system]

[Insert screenshot or diagram here]

Figure 6. Screenshot of the repository page showing files and commit history

EVALUATION CRITERIA

For reference: this is how the project will be graded.

Criteria	Description	Points
Business Process	Realistic process clearly described; problems and proposed system well defined	15

Data Structures and Algorithms Applied	At least five course topics, correctly explained and justified for the business process	25
System Flowchart	Complete, correct symbols, matches the program	10
Output	All features shown working, including invalid input handling	15
Performance Analysis	Correct theoretical complexity; sound experimental method; meaningful comparison	25
Documentation and Appendices	Complete code, team composition, working GitHub repository; clear writing	10
Total		100